

SPHINCS⁺: A Stateless Hash-Based Post-Quantum Cryptographic Algorithm

TIEMAR BERHE, XHELION DEDELLI, ANNA IVANYTSKA, ASSAN JALLOW, and SARAH SUMRALL

In today's cybersecurity landscape, it is essential to consider protection against quantum computing attacks at every stage of system development. One approach to such protection is the SPHINCS⁺ stateless, hash-based post-quantum signature algorithm. While SPHINCS⁺ is not the fastest signature scheme, its security relies on well-studied cryptographic hash functions, in contrast to newer post-quantum algorithms that may be vulnerable to unknown attacks. This report provides an explanation of SPHINCS⁺'s core components and their functionalities, discusses its performance and security metrics, additional SPHINCS⁺ instantiations, lists advantages and disadvantages of SPHINCS⁺, and highlights improvements over its predecessor.

1 Introduction

Large-scale Quantum Computers (QCs) do not exist right now, but they might exist in the future. If an attacker can obtain a large-scale Quantum Computer, they can break asymmetric algorithms that belong to RSA [1], discrete logarithm [2], and ECC [3] families. To deal with this, new public-key algorithms that are not vulnerable to QCs must instead be used. Such algorithms are called post-quantum cryptography. There are two main applications of public-key cryptography: digital signatures and key exchange/encryption. The three most promising post-quantum cryptography families and their use cases are shown in Table 1. SPHINCS⁺ is a hash-based PQC algorithm that is used for digital signatures [4].

Table 1. Post Quantum Cryptography families and their use cases

Algorithm family	digital signatures	key exchange
Lattice-based	+	+
Code-based		+
Hash-based	+	

SPHINCS⁺ is considered a Stateless Hash Based Post Quantum Cryptographic Algorithm. Like its predecessor: SPHINCS, SPHINCS⁺ is a hash-based signature algorithm created to defend systems against attacks from quantum computers, without maintaining its state [5]. It combines multiple cryptographic primitives, including FORS, WOTS⁺, and XMSS trees, arranged in a hypertree structure. Each component is essential to the SPHINCS⁺ algorithm and is discussed in this report, including: an analysis of the improvements from SPHINCS to SPHINCS⁺; an overview of the different SPHINCS⁺ instantiations; a description of SPHINCS⁺'s security; and an analysis on SPHINCS⁺'s performance.

1.1 SPHINCS vs. SPHINCS⁺

SPHINCS⁺ implemented five improvements from the original SPHINCS algorithm, which include: **Multi-Target Attack Protection**, **Treeless WOTS+ Public Key Compression**, **FORS**, **Verifiable Index Selection**, **Optional Deterministic Signing**, and **SPHINCS⁺-simple and SPHINCS⁺-Robust** [6]. In this section, each improvement will be discussed in detail.

The first improvement implemented in SPHINCS⁺ was **Multi-Target Attack Protection**. It was discovered multi-target attacks could target and exploit essential preimage resistance properties within the SPHINCS algorithm [6]. These attacks could range from inverting the hash function at a given target, to breaking the hash function's second-preimage resistance [6]. Therefore, SPHINCS⁺ utilizes keyed hash functions to safeguard against multi-target attacks. Each key in the keyed hash function serves two purposes: (1) as an address to pinpoint where its hash is used and (2) as a mechanism to keep each hash separate from other hashes [7]. Thus, keyed hash functions make it challenging for the adversary to preform a multi-target attack because each potential target "cryptographically independent" from each other [7].

The second improvement provided by SPHINCS⁺ was the **Treeless WOTS+ Public Key Compression**. While the original SPHINCS algorithm utilized L-Trees or "tree-based compression" to compress the WOTS+ public key into a single hash, SPHINCS⁺ "compresses the end nodes of the WOTS+ hash chains with a single call to a tweakable hash function" [6, 8]. Thus, this means that even with longer input, the SPHINCS⁺ algorithm is still more secure and efficient than SPHINCS [6, 8].

The third improvement implemented in SPHINCS⁺ was the replacement of HORST with **FORS** [6]. There was a significant vulnerability in how HORST handled secret key values [9]. HORST mapped its messages to subsets which only selected "from the **same single set** of secret key values" [6, 9]. This is concerning because the reuse of the same secret key set can make it easier for the adversary to hijack the signature "by trying to find a message where multiple chunks match in value, thus decreasing the number of private key chunks that need to be revealed" [6, 9, 10]. In SPHINCS⁺, FORS patches this vulnerability because it utilizes separate secret value sets per mapped subset [6, 9]. Even if a message maps multiple times to the same subset, the signature now contains multiple different, secret values [6, 9]. Therefore, FORS and SPHINCS⁺ is more secure than HORST and SPHINCS.

The fourth improvement provided by SPHINCS⁺ was the **Verifiable Index Selection**. Like the previous improvement, Verifiable Index Selection serves as a way to safeguard against Multi-Target Attacks in SPHINCS [6, 9]. With HORST and SPHINCS, the subset/index was pseudo-randomly generated with no verification mechanism, making it easier for adversaries to commit Multi-Target Attacks [6, 9]. They committed these attacks by using the random subset to reveal the HORST key pair and forge a signature with the reused secret key set [6, 9]. Yet, with SPHINCS⁺, there is no random, unverifiable subset that can be used to forge a signature [6, 9]. In SPHINCS⁺, each subset is fixed with the message digest which makes it impossible for adversaries select subsets and therefore commit Multi-Target Attacks [6, 9].

The fifth improvement implemented by SPHINCS⁺ was the requirement to make **deterministic signing optional**. Since deterministic signing can provide significant risks of side-channel attacks in certain technologies, SPHINCS⁺ mitigates these risks by using a randomizer that makes deterministic signing optional [6].

The final improvement provided in SPHINCS⁺ was the instantiation **SPHINCS⁺-simple**, which implements a different version of the tweakable hash functions [6]. The simple instantiations were developed to improve upon the **robust** instantiations which were considered the already established, standard implementations of tweakable hash functions in SPHINCS⁺ [6]. The simple instantiation improves the speed of the algorithm by "omitting the use of bitmasks" [6]. Yet, this omission results in significant drawbacks in SPHINCS⁺'s security [6]. Thus, the robust instantiations are still available to provide more security within the algorithm at a slower speed [6].

2 Components

SPHINCS⁺ employs several structured components that work together to produce a hash-based signature. This section introduces these components in a self-contained manner. The main building blocks are the FORS few-time signature system, the WOTS⁺ one-time signature scheme, Merkle trees that authenticate the outputs of these primitives, and hypertrees that extend the authentication structure across multiple layers. The parameters of SPHINCS⁺ are defined in table 2.

2.1 The Winternitz One-Time Signature Plus (WOTS+) Scheme

The Winternitz One-Time Signature scheme (WOTS⁺) [6, 11] forms the one-time signature primitive in SPHINCS⁺. It is a hash-based construction that offers short signatures and verification. In SPHINCS⁺, WOTS⁺ does not sign the message directly. It signs the root output of the FORS mechanism, section 2.3 or, at upper hypertree layers, the root of the XMSS subtree below. Thus, each WOTS⁺ instance authenticates a single n -byte string and links one tree layer to the next.

WOTS⁺ uses the parameters n , w , and the derived values ℓ_1 , ℓ_2 , and $\ell = \ell_1 + \ell_2$. The string to be signed is first expanded into a sequence of ℓ_1 base- w digits [6]. A checksum prevents an attacker from reducing work by choosing digits that require fewer hash-chain steps. The checksum is expressed in base w and contributes ℓ_2 additional digits. The concatenation of both parts yields a vector of length ℓ , with each element in the range $\{0, \dots, w - 1\}$.

Key generation in WOTS⁺ proceeds by sampling ℓ secret values of length n . Each value is processed through a hash chain of length $w - 1$, with masking in every step for side-channel protection. The final outputs of all chains form the WOTS⁺ public key. Signing requires only partial evaluation of the chains. If the base- w expansion contains the digit m_i at position i , then the signer outputs the intermediate value obtained after m_i applications of the hash function. The verifier completes each chain up to length $w - 1$ and reconstructs the public key. A comparison with the WOTS⁺ leaf inside the XMSS tree then confirms correctness.

Symbol	Description	Notes / Formula
WOTS⁺ Parameters		
n	Security parameter; byte-length of keys and hashes	Common across components
w	Winternitz parameter	$w \in \{4, 16, 256\}$
ℓ	Total number of chains	$\ell = \ell_1 + \ell_2$
ℓ_1	Chains for message part	$\ell_1 = \frac{8n}{\log_2(w)}$
ℓ_2	Chains for checksum	$\ell_2 = \left\lceil \frac{\log_2(\ell_1(w-1))}{\log_2(w)} \right\rceil + 1$
XMSS Parameters		
h'	Height of the Merkle tree	Tree has $2^{h'}$ leaves
n	Length of each node (bytes)	Same n as WOTS ⁺
SIG_{XMSS}	XMSS signature	Contains one WOTS ⁺ sig + auth path
Hypertree (HT) Parameters		
h	Total hypertree height	Split into d layers
d	Number of tree layers	Must divide h exactly
h/d	Height of each XMSS subtree	Used uniformly across layers
FORS Parameters		
k	Number of FORS trees	Determines number of indices
t	Leaves per FORS tree	Must satisfy $t = 2^a$
a	Height of each FORS tree	$a = \log_2(t)$

Table 2. Notation and parameters used for WOTS⁺, XMSS, hypertrees, and FORS in SPHINCS⁺.

2.2 XMSS and Hypertrees

XMSS is a hash-based signature scheme that authenticates signatures through a Merkle tree. It uses a tree of height h' with $2^{h'}$ leaves, and each leaf contains a WOTS⁺ public key. An XMSS signature supplies the corresponding WOTS⁺ signature and an authentication path of h' nodes. This path enables the verifier to compute the tree root from the signed leaf.

SPHINCS⁺ requires many WOTS⁺ signatures and therefore cannot depend on a single XMSS tree. The design addresses this requirement through a hypertree with d XMSS layers, each of height h/d . Each layer signs the root produced by the layer below. The final root at the top layer becomes the SPHINCS⁺ public key. A SPHINCS⁺ signature contains one WOTS⁺ signature and one XMSS authentication path for every layer. The verifier reconstructs each intermediate root in sequence until the top root is reached. The hypertree therefore establishes a chain of authenticated roots, beginning with the FORS output and progressing through each XMSS layer, until the process reaches the final root that serves as the SPHINCS⁺ public key.

2.3 Forest of Random Subsets (FORS)

FORS is a few-time signature component within SPHINCS⁺ based on a collection of small Merkle trees and constitutes the first stage of the signing process. The root of each FORS tree forms part of the FORS public key. SPHINCS⁺ uses FORS to sign the hashed message. The message digest is interpreted as k indices, each in the range $\{0, \dots, t-1\}$. Each index selects one leaf from the corresponding FORS tree. These leaves represent the few-time secret key elements for the current message. The signer outputs the selected leaf values and the authentication paths required to reach the roots of the k FORS trees. Using these values, a signer and a verifier can reconstruct each tree root and apply the scheme's hash function to combine the k roots into a single n -byte value. This hashed value is referred to as the FORS root. It completes the FORS stage of the signing process and is treated as the message for the subsequent WOTS⁺ signature.

2.4 SPHINCS⁺ Private Key

The SPHINCS⁺ private key contains the secret material required to derive all components of the signing process, including the FORS secret values, WOTS⁺ secret chains, and the authentication nodes in the hypertree. In its compressed form, the private key consists of three main elements: the secret seed SK_{seed} , the pseudorandom function key SK_{PRF} , and the public key seed PK_{seed} .

The seed SK_{seed} deterministically expands all secret key values on demand. These values are generated through a pseudorandom function indexed by an address that identifies the FORS leaf, WOTS⁺ chain, or XMSS node. This approach enables SPHINCS⁺ to remain stateless because it removes the need to store large trees or chain states.

The second component, SK_{PRF} , produces the per-signature randomness, R , which depends on the secret key and the message. This value prevents side-channel attacks by ensuring that signatures do not leak secret key material that is correlated across multiple signing operations. Because SPHINCS⁺ relies purely on hash-based constructions (tweakable hash functions), the pseudorandom function is implemented using the same permutation or compression function as the rest of the scheme, leading to a simple and uniform design. The final private-key element, PK_{seed} , supports deterministic computation of public-key material, including WOTS⁺ verification keys and XMSS nodes, while remaining domain-separated from private-key expansion. It is the public seed included in the public key and ensures that all public-side computations are consistent and reproducible across signers.

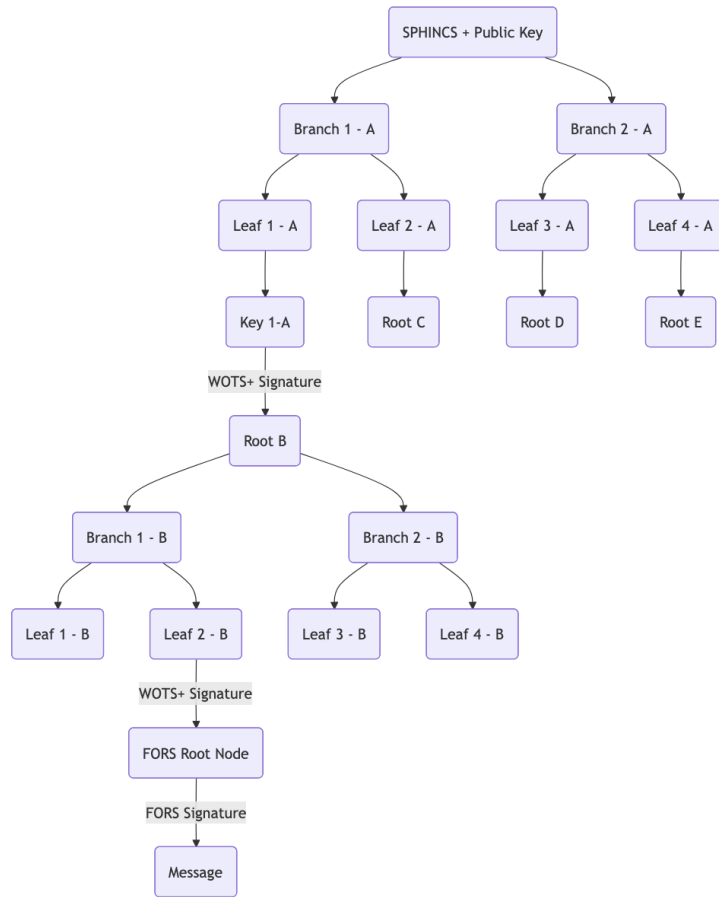


Fig. 1. Ethan Rahn's SPHINCS⁺ diagram[10]

Together, these three components allow the signer to derive all parts of the authentication structure without storing large or stateful secret trees. During signing, all the elements interact as follows: The signer computes R and uses it, together with the public key and the message, to compute the message digest and parse it to the FORS indices. These indices select k FORS trees. For each tree, the signer derives the required secret leaf from SK_{seed} , outputs the selected secret leaf and its authentication path. The k FORS roots are compressed into a single n -byte value. This value becomes the input to a WOTS⁺ signature at the bottom layer of the hypertree. WOTS⁺ chain steps use per-step masks derived from PK_{seed} and address fields to harden chains. The resulting WOTS⁺ public key forms a leaf in the corresponding XMSS tree, whose authentication path is then output. This procedure repeats

across all hypertree layers until the top-root matches the public key. This integrated process shows how FORS, WOTS⁺, and the hypertree collectively produce a SPHINCS⁺ signature using only seed-derived secret material.

The design emphasizes determinism, statelessness, and minimal memory usage, motivations behind SPHINCS⁺ as a conservative hash-based post-quantum signature scheme. Because all secret values are generated from seeds through a pseudorandom expansion mechanism, the private key remains compact while still supporting a large, hierarchical authentication structure extending through FORS, WOTS⁺, and the hypertree.

2.5 Addressing Scheme

SPHINCS⁺ uses a 32-byte address structure to provide domain separation across all hash function calls in the signature scheme. The address uniquely identifies the context of each hash invocation, preventing multi-target attacks and ensuring independence between different parts of the tree structure. The first 4 words (words 0–3) contain the layer address and tree address, identifying the position within the hypertree. The fifth word (indexed word 4) specifies the address type, which determines the interpretation of the remaining 3 words (words 5–7). SPHINCS⁺ defines five distinct address types: WOTS⁺ Hash addresses, WOTS⁺ Public Key Compression addresses, Main Tree Hash addresses (identify nodes during Merkle tree construction in the XMSS layers); FORS Tree addresses specify positions within the individual Merkle trees of the FORS structure; and FORS Public Key Compression addresses are used when combining multiple FORS tree roots into a unified public key. Table 3 summarizes the SPHINCS⁺ addressing scheme.

Type	Value	Fields (bytes 16–31)	Purpose
<i>Common Fields (bytes 0–15): Layer Address Tree Address Type</i>			
WOTS ⁺ Hash	0	16–19: Keypair Address 20–23: Chain Address 24–27: Hash Address 28–31: Padding	Hash computation within WOTS ⁺ signature chains. Chain address specifies which of ℓ chains; hash address specifies position in chain (0 to $w - 1$).
WOTS ⁺ PK	1	16–19: Keypair Address 20–31: Padding	Compress ℓ WOTS ⁺ chain endpoints into single n -byte public key using T_ℓ .
Tree Hash	2	16–19: Padding 20–23: Tree Height 24–27: Tree Index 28–31: Padding	Compute inner nodes in XMSS Merkle trees. Tree height 0 is leaf level; tree index is horizontal position at that height.
FORS Tree	3	16–19: Keypair Address 20–23: Tree Height 24–27: Tree Index 28–31: Padding	Hash nodes within FORS trees. Keypair address identifies which of k FORS trees in the instance.
FORS PK	4	16–19: Keypair Address 20–31: Padding	Compress k FORS tree roots into single n -byte FORS public key using T_k .

Table 3. SPHINCS⁺ address types and their structure (32 bytes total, type-specific fields shown)

3 Instantiations

This section discusses the instantiations for SPHINCS⁺. SPHINCS⁺ can be considered as a signature template, which means that by selecting different cryptographic function families or parameter values, different instantiations of this algorithm can be implemented [6]. However, there exist several trade-offs between security, sizes, and speed controlled by the different parameters.

3.1 Influence of Parameters on Security and Performance

All parameters in SPHINCS⁺ can take different positive integer values, which can influence the performance and the security of the algorithm. The **security** of SPHINCS⁺ mainly depends on the security parameter n , which defines the hash output length; larger values of n provide stronger protection [12]. The hypertree height h also enhances security by reducing the chance of key reuse, while in the FORS scheme, increasing k strengthens security and smaller t requires a higher k to maintain it. **Performance** is influenced by several parameters: a higher w or larger t slows signing due to longer computations, whereas more layers d

and smaller k improve speed. **Signature size** grows with larger n and h , but decreases with higher w . In FORS, increasing k or decreasing t leads to larger signatures. The full influence of these parameters is summarized in table 6. Overall, tuning these parameters requires balancing security, performance, and compactness [12].

3.2 Proposed Parameter Sets

According to [6], SPHINCS⁺ defines six parameter sets across three security levels, 128, 192, and 256 bits [13], with each offered in size-optimized (“s”) and speed-optimized (“f”) versions. These were selected using a Sage script to balance signature size and signing time, avoiding extreme trade-offs, like very small size but slow signatures [12]. The selection ensures practical signing times, around few seconds, while maintaining strong security. The proposed six parameter sets from third submission are presented in table 7.

The scheme supports three hash function families, SHA-256, SHAKE256, and Haraka, each available in a ‘robust’ and ‘simple’ version. The robust versions include an additional step by XORing the input with random bitmasks for stronger security, while the simple versions skip this step for faster performance [14]. Together, these choices result in 36 SPHINCS⁺ instantiations, covering a wide range of security and performance needs.

3.3 Instantiations of Hash Functions

Starting with **SHAKE256**, it is based on the Keccak sponge construction, offering flexible and efficient variable-length outputs, making it highly performant and suitable for mask generation. Next, **SHA-256** provides a well-known security foundation but with slightly lower performance due to its fixed output size. To improve performance, SPHINCS⁺ uses padding and compression on the input parameters of these SHA-256 Hash functions, along with truncating the output if the required length $n < 32$ -bytes [6]. Lastly, **Haraka** is a lightweight, short-input of 256 and 512 bits hash function, optimized for speed. It is faster than previous Hash function families, however, it is not yet NIST-approved and provides weaker security for small output sizes [15]. For Haraka the security level is limited to 2 for $n = 24$ or $n = 32$.

4 Security

The security of SPHINCS⁺ relies on the properties of the functions used to instantiate all the cryptographic function families [6]. In the following, it is assumed that there do not exist any structural attacks against the used functions SHA-256, SHAKE256, and Haraka. These properties are discussed below.

The **Post-Quantum Distinct-Function Multi-Target Second-Preimage Resistance (PQ-DM-SPR)** guarantees that, even when an adversary has access to many distinct hash instances, the probability of finding a valid second preimage remains negligible. This ensures that no efficient quantum attacker can forge a signature by exploiting collisions or reused structures within SPHINCS⁺’s layered architecture (FORS, WOTS+, and the hypertree) [16].

The **Post-Quantum Interleaved Target Subset Resilience (PQ-ITSR)** property strengthens this protection against cross-instance correlations, preserving the independence of message subsets across different FORS instances and preventing hybrid subset-target attacks [16, 17]. Moreover, the **Pseudorandomness** of the function families (PRF, PRFmsg, PRFbm) guarantees that seeds, masks, and message randomness are computationally indistinguishable from random values, even under quantum-accessible random oracle conditions.

Quantitatively, SPHINCS⁺ provides NIST security levels 1, 3, and 5, corresponding to 128, 192, and 256-bit post-quantum strength [6]. Empirical and theoretical analyses estimate that a successful quantum attack, like Grover or multi-target preimage search, would require between 2^{120} and 2^{250} quantum operations depending on the chosen parameters [18]. The probability of a successful forgery remains below 2^{-128} for Level 1 and decreases to approximately 2^{-256} for Level 5.

5 SPHINCS⁺ Performance

SPHINCS⁺ was designed to provide post-quantum security without relying on algebraic hardness assumptions, resulting in strong resistance to quantum attacks but at the cost of comparatively large signatures and relatively expensive signing operations. The performance of the scheme is mainly shaped by three components: the FORS mechanism, the WOTS+ one-time signature layer, and the hypertree configuration. The FORS structure contributes significantly to the overall signature size because it includes multiple authentication paths; however, it remains computationally efficient due to the relatively small height of its binary subtrees.

In contrast, WOTS⁺ is more computationally demanding, as it relies on iterative hashing along multiple chains, making it one of the most resource-intensive components of the signing process.

The hypertree mitigates these costs by distributing the authentication workload across several smaller XMSS layers rather than relying on a single monolithic Merkle tree. This structure reduces both signature size and computational overhead while maintaining statelessness and robust security guarantees. Performance enhancements introduced in the Round 3 revision of SPHINCS⁺ demonstrate meaningful improvements: for example, the 128S and 192S parameter sets achieved reduced signature sizes and faster verification times, whereas the 128F and 256F variants experienced notable increases in signing speed up to approximately 24% for the 128F parameter set, without altering the underlying security assumptions or significantly impacting verification performance. These optimizations illustrate the careful balance between signature size, verification efficiency, and signing speed that characterizes SPHINCS⁺ as a practical and conservative stateless hash-based post-quantum signature scheme.

6 Advantages and Limitations

The clear drawback of SPHINCS⁺ is **signing speed and signature size**. SPHINCS⁺ is not the smallest or fastest signature scheme. However, it is possible to tweak SPHINCS⁺ to achieve either faster signing or smaller signature size.

In contrast to other post-quantum crypto schemes, the security of SPHINCS⁺ is solely based on assumptions about the used hash function. Moreover, a collision attack against the hash function does not suffice to break the security of SPHINCS⁺. The cryptographic community has a good understanding of hash-function security, especially after the recent SHA3 competition. This is in contrast to the relatively new problems used in other areas of post-quantum cryptography. SPHINCS⁺ uses the basic hash function as building block many times. Any speedup to implementations of SHA-256, SHAKE256 or Haraka directly benefits the SPHINCS⁺ speed. The most efficient attacks known against SPHINCS⁺ are easy to state and analyze. Another advantage of SPHINCS⁺ is the small size of the keys, in particular the public-key size [6].

7 Conclusion

The SPHINCS⁺ Algorithm is a stateless hash-based digital signature scheme designed to provide security against post-quantum computing attacks. This report presented an analysis of SPHINCS⁺ explaining its core components, including FORS, WOTS⁺, XMSS trees, and the hypertree structure. It also discussed these components' functions, SPHINCS⁺ security, improvements made over SPHINCS, and additional SPHINCS⁺ instantiations.

References

- [1] R. L. Rivest, A. Shamir, and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, 1978.
- [2] W. Diffie and M. E. Hellman, "New directions in cryptography," in *IEEE Transactions on Information Theory*, vol. 22, no. 6. IEEE, 1976, pp. 644–654.
- [3] V. S. Miller, "Use of elliptic curves in cryptography," in *Advances in Cryptology—CRYPTO 85 Proceedings*. Springer, 1986, pp. 417–426.
- [4] C. Paar, "Lecture 23: Introduction to hash-based post-quantum cryptography (pqc)." 2025. [Online]. Available: <https://www.youtube.com/watch?v=aw8K-ljRpIE>
- [5] Codebyankita, "Sphincs+: A comprehensive guide to post-quantum signatures in blockchain," Oct. 2024. [Online]. Available: <https://medium.com/@ankitacode11/sphincs-a-comprehensive-guide-to-post-quantum-signatures-in-blockchain-7c6e0bbfd4aa>
- [6] J.-P. A. et al., "Sphincs+ submission to the nist post quantum project, v.3," p. 62, Oct. 2020. [Online]. Available: <https://sphincs.org/>
- [7] ChatGPT, "Keyed hash functions security," Oct. 2025. [Online]. Available: <https://chatgpt.com/share/6901f6fe-fed0-800d-b059-d22bf66775af>
- [8] —, "Tree-less wots+ compression," Oct. 2025. [Online]. Available: <https://chatgpt.com/share/69047380-3a80-800d-812e-2148e9d0fa97>
- [9] J.-P. Aumasson and G. Endignoux, "Clarifying the subset-resilience problem," 2017. [Online]. Available: <https://eprint.iacr.org/2017/909>
- [10] E. Rahn, "Sphincs+ - step by step," Dec. 2023. [Online]. Available: <https://er4hn.info/blog/2023.12.16-sphincs-plus-step-by-step/>
- [11] A. Hülsing, D. Butin, S. Gazdag, J. Rijneveld, and A. Mohaisen, "XMSS: eXtended Merkle Signature Scheme," RFC 8391, May 2018. [Online]. Available: <https://www.rfc-editor.org/info/rfc8391>
- [12] S. Kölbl and J. Philipoom, "A note on SPHINCS+ parameter sets," 2022. [Online]. Available: <https://eprint.iacr.org/2022/1725>
- [13] M. Mema. (2019) Nist pqc security strength categories (1–5) explained. PostQuantum – Quantum Computing. [Online]. Available: <https://postquantum.com/post-quantum/nist-pqc-security-categories/>
- [14] J.-P. Aumasson and G. Endignoux, "Improving stateless hash-based signatures," Cryptology ePrint Archive, Paper 2017/933, 2017. [Online]. Available: <https://eprint.iacr.org/2017/933>
- [15] S. Kölbl, M. M. Lauridsen, F. Mendel, and C. Rechberger, "Haraka v2 – efficient short-input hashing for post-quantum applications," *IACR Transactions on Symmetric Cryptology*, vol. 2016, no. 2, p. 1–29, Feb. 2017. [Online]. Available: <https://tosc.iacr.org/index.php/ToSC/article/view/563>
- [16] A. Hülsing, J. Rijneveld, and F. Song, "Mitigating multi-target attacks in hash-based signatures," in *Public Key Cryptography – PKC 2016*, ser. Lecture Notes in Computer Science, C.-M. Cheng, K.-M. Chung, G. Persiano, and B.-Y. Yang, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2016, pp. 387–416.
- [17] A. Genêt, "On protecting sphincs+ against fault attacks," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2023, no. 2, p. 80–114, Mar. 2023. [Online]. Available: <https://tches.iacr.org/index.php/TCHES/article/view/10278>
- [18] R. Perlner, J. Kelsey, and D. Cooper, "Breaking category five SPHINCS+ with SHA-256," Cryptology ePrint Archive, Paper 2022/1061, 2022. [Online]. Available: <https://eprint.iacr.org/2022/1061>

A Appendix

Table 4. Number of function calls per operation (single calls to Hmsg, PRFmsg, Tk omitted).

	F	H	PRF	T_{len}
Key Generation	$2^{h/d} w \text{len}$	$2^{h/d} - 1$	$2^{h/d} \text{len}$	$2^{h/d}$
Signing	$kt + d(2^{h/d})w \text{len}$	$k(t - 1) + d(2^{h/d} - 1)$	$kt + d(2^{h/d}) \text{len}$	$d2^{h/d}$
Verification	$k + d w \text{len}$	$k \log t + h$	–	d

Table 5. Key and signature sizes in bytes.

Type	SK	PK	Signature
Size	$4n$	$2n$	$(h + k(\log t + 1) + d \cdot \text{len} + 1)n$

Table 6. Overview of how SPHINCS⁺ parameters influence security, performance, and signature size.

Parameter	Effect on Security	Effect on Performance	Effect on Signature Size
n	Higher $n \rightarrow$ higher security	Slightly slower	Larger signatures
h	Taller tree $\rightarrow$ higher security	Slower signing	Larger signatures
d	No effect	More layers $\rightarrow$ faster signing	Slightly smaller signatures
w	No effect	Higher $w \rightarrow$ slower signing	Higher $w \rightarrow$ smaller signatures
k	Higher $k \rightarrow$ higher security	Slower signing	Larger signatures
t	Smaller $t \rightarrow$ less secure unless k increased	Larger $t \rightarrow$ faster signing	Larger $t \rightarrow$ slightly smaller signatures

Table 7. SPHINCS⁺ parameter sets (n in bytes, $\log_2 t = a$, sig in bytes)

	n	h	d	$\log_2 t$	k	w	Bitsec	Sec level	Sig (bytes)
SPHINCS ⁺ -128s	16	63	7	12	14	16	133	1	7 856
SPHINCS ⁺ -128f	16	66	22	6	33	16	128	1	17 088
SPHINCS ⁺ -192s	24	63	7	14	17	16	193	3	16 224
SPHINCS ⁺ -192f	24	66	22	8	33	16	194	3	35 664
SPHINCS ⁺ -256s	32	64	8	14	22	16	255	5	29 792
SPHINCS ⁺ -256f	32	68	17	9	35	16	255	5	49 856